

Logos Experiment 3 Report

RMSNorm Normalization vs Original LayerNorm Baseline

Experiment	logos_exp03_rmsnorm
Core change	LayerNorm replaced with RMSNorm
Architecture family	Decoder-only GPT-style causal language model
Training budget	1500 iterations on OpenWebText subset
Main conclusion	RMSNorm did not improve this fixed-budget Logos setup

Scientific purpose

The goal of this experiment was to test whether replacing LayerNorm with RMSNorm improves the capability-resource tradeoff of the Logos decoder-only language model. The experiment intentionally kept the architecture, data, tokenizer, training budget, activation function, batch size, and optimizer settings unchanged except for the normalization layer. This makes the result suitable for controlled comparison against the original baseline.

Key result

RMSNorm slightly reduced the parameter count, but it produced a slightly worse validation loss, worse perplexity, slower training throughput, higher measured VRAM, slower generation latency, and higher estimated energy usage than the original LayerNorm baseline.

1. Experiment Setup

The model remained a GPT-style decoder-only causal language model. It was trained for next-token prediction, where the model predicts each next token from previous tokens only. This is the same language modeling objective used by GPT/LLaMA-style causal models.

Category	Value
Run name	logos_exp03_rmsnorm
Dataset	OpenWebText subset
Samples	10,000
Tokenizer	GPT-2 tokenizer
Objective	Next-token prediction
Architecture	Decoder-only Transformer
Layers	8
Attention heads	8
Embedding dimension	256
Context length	256 tokens
Activation	ReLU
Normalization	RMSNorm
Weight tying	False
Optimizer	AdamW
Learning rate	3e-4
Weight decay	0.1
Max iterations	1500
Batch size	64
Mixed precision	Enabled on CUDA

Only one experimental variable was changed:

LayerNorm -> RMSNorm

2. Final RMSNorm Results

Metric	RMSNorm Result
Parameters	32,155,473
Trainable parameters	32,155,473
Best iteration	1500
Best validation loss	5.7804
Final train loss	5.5960
Final validation loss	5.7722
Final validation perplexity	321.24
Checkpoint size	122.69 MB
Peak VRAM	11.11 GB
Training time	169.97 sec = 2.83 min
Training speed	~145,355 tokens/sec
Generation speed	74.02 tokens/sec
Generation latency	13.51 ms/token
Average GPU power	246.59 W
Estimated GPU energy	11.50 Wh

Training behavior

The validation loss continued to improve throughout the 1500-iteration fixed-budget run. The best checkpoint was saved at iteration 1500, meaning the RMSNorm model had not clearly converged by the end of the run. This is consistent with the original baseline and means the result should be interpreted as a fixed-budget comparison rather than a fully converged model comparison.

Iteration	Train loss	Validation loss	Perplexity
0	10.8630	10.8647	52298.62
150	6.9678	7.0144	1112.55
300	6.3951	6.4715	646.45
600	5.9792	6.0874	440.27
900	5.7541	5.8915	361.96
1200	5.6265	5.7991	330.01
1500	5.5995	5.7804	323.90

3. Comparison Against Original Baseline

The original baseline used LayerNorm with ReLU and no weight tying. The RMSNorm experiment kept ReLU and no weight tying, making the normalization change the primary experimental factor.

Metric	Baseline LayerNorm	RMSNorm	Outcome
Parameters	32,159,825	32,155,473	4,352 fewer
Final val loss	5.7634	5.7722	Worse
Final perplexity	318.42	321.24	Worse
Checkpoint size	122.71 MB	122.69 MB	Almost same
Peak VRAM	10.84 GB	11.11 GB	Worse
Training time	156.42 sec	169.97 sec	Slower
Training speed	~158k tok/s	~145k tok/s	Worse
Generation latency	10.20 ms/token	13.51 ms/token	Worse
Energy	10.78 Wh	11.50 Wh	Worse

Parameter reduction calculation

The only clear structural benefit from RMSNorm was a small parameter reduction:

$$32,159,825 - 32,155,473 = 4,352 \text{ fewer parameters}$$

This reduction is too small to matter practically because the vocabulary projection and embedding layers dominate the total parameter count. In this model, replacing LayerNorm with RMSNorm saves only the bias terms from the normalization layers.

4. Scientific Interpretation

For this fixed-budget Logos setup, RMSNorm is not a successful improvement over LayerNorm. Although RMSNorm is often used in modern LLaMA-style models, the result here shows that an isolated normalization replacement does not automatically improve performance. Architecture changes are context-dependent and must be measured rather than assumed.

Observation	Interpretation
Validation loss increased from 5.7634 to 5.7722	The RMSNorm model learned slightly worse under the same budget.
Perplexity increased from 318.42 to 321.24	Prediction confidence/quality slightly decreased.
Training speed dropped from about 158k to 145k tokens/sec	This implementation did not provide a speed advantage.
VRAM rose from 10.84 GB to 11.11 GB	Measured memory behavior was not improved.
Energy rose from 10.78 Wh to 11.50 Wh	Efficiency per watt was worse in this run.

Conclusion statement

In the current Logos configuration, LayerNorm remains better than RMSNorm. RMSNorm did not deliver a meaningful parameter advantage and reduced the observed capability-resource efficiency under the same training budget.

5. Notes for Research Log

- This was a clean one-variable experiment: LayerNorm was replaced with RMSNorm while keeping ReLU, no weight tying, same data, same tokenizer, same model size target, same optimizer, and same training budget.
- The experiment proves why controlled testing matters: a technique that is common in modern LLMs did not improve this small Logos baseline when applied in isolation.
- The result should not be interpreted as "RMSNorm is bad" in general. It only means RMSNorm was not beneficial in this exact configuration and 1500-iteration budget.
- The next architecture experiment should test a stronger FFN change such as SwiGLU, because the normalization-only change produced minimal benefit.

6. Limitations

Limitation	Effect on interpretation
Short training budget	The model was trained only for 1500 iterations, so longer-run behavior may differ.
Small dataset subset	Only 10,000 OpenWebText samples were used.
Single seed	The comparison may contain random-seed variance.
No hyperparameter retuning	RMSNorm might need slightly different learning rate or regularization.
Custom RMSNorm implementation	A fused/optimized implementation might show different speed and memory behavior.

7. Recommended Next Experiment

The next recommended experiment is a SwiGLU feed-forward network. This tests a more substantial modern LLM-style architectural change than normalization alone.

Experiment 4: Replace baseline FFN with SwiGLU FFN

Expected research question: Does a gated feed-forward network improve validation loss and perplexity enough to justify any extra parameter, VRAM, or latency cost?

One-line summary

RMSNorm slightly reduced the parameter count but did not improve the Logos baseline; LayerNorm remains the better normalization choice for this fixed-budget run.